

第 08 章 传统权限、umask 与特殊权限

从访问主体到路径逐级求值：把 owner/group/other、文件与目录 `rx`、创建屏蔽和特殊位放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：权限不是一串孤立数字。每一次访问都由一个具有实际凭据的进程发起，内核逐级解析路径、为每个对象选中 owner/group/other 中唯一一类，再依据具体动作检查文件或目录的权限；新对象还要经过 `umask` 与父目录特殊位的影响。

- 01确认真实主体有效 UID、有效 GID 与附加组
- 02逐级解析路径每一级父目录都可能因缺少 `x` 阻断
- 03选择权限类owner / group / other 只命中一类，不叠加、不回退
- 04把动作映射到权限文件与目录的 `rwX` 含义不同
- 05处理创建与特殊位 `umask`、`setuid`、`setgid`、`sticky`
- 06用真实用户验收静态状态、正向功能与负向隔离分层证明

专题地图

知识专题	传统 DAC 如何选择 owner/group/other
知识专题	文件与目录的 <code>rwX</code> 及父目录控制
操作专题	使用 <code>ls/stat/id/namei</code> 建立证据
操作专题	<code>chmod</code> 、 <code>chown</code> 、 <code>chgrp</code> 的最小变更
知识/操作	<code>umask</code> 计算、当前状态与持久验证
知识/操作	<code>setuid</code> / <code>setgid</code> / <code>sticky</code> 与组协作目录
诊断专题	路径不可达与递归修改风险
经典任务	协作目录；文件可读但路径不可达

阅读时持续回答

1. 当前真正发起操作的进程是谁？
2. 路径中第一处可能阻断的位置在哪里？
3. 当前对象会选中 owner、group 还是 other？
4. 原始动作需要文件权限还是父目录权限？
5. 新对象的请求模式是什么，`umask` 清除了哪些位？
6. 特殊位作用于可执行文件还是目录？
7. 当前证据能证明什么，不能证明什么？
8. 下一条最有区分度的证据是什么？

第 08 章 · 正文

用户看到 `Permission denied` 时，最容易犯的错误是盯着最终文件反复执行 `chmod`。Linux 的传统访问控制不是“看一行 `ls -l` 就结束”：访问请求由一个具有有效 UID、有效 GID 和附加组的进程发起；内核先逐级解析路径，再为每个相关 inode 选择 owner、group 或 other 中唯一一类权限；创建、删除和重命名还主要受父目录控制；新对象的权限则取决于程序请求的模式、进程的 `umask` 以及父目录是否具有 `setgid` 等属性。

本章沿着“主体 → 路径 → 权限类 → 动作 → 创建规则 → 功能证据”推进。前一章已经建立用户、组与会话身份，本章不重复账号生命周期；ACL 留给第 09 章，sudo 授权留给第 10 章，SELinux 留给第 28/29 章。这里使用 `sudo -u` 只为建立指定普通用户的验证进程。

概念

DAC (Discretionary Access Control, 自主访问控制) 是由对象 owner、group 与传统 mode bits 共同表达的访问控制层。它回答的是“这个进程按当前身份，能否对这个 inode 完成某个具体动作”，而不是抽象地判断“某用户有没有权限”。观察 DAC 时，要把进程凭据、路径分量和 inode 元数据放在一起。

概念

owner / group / other 是三个互斥的权限类别。内核先比较有效 UID；未命中 owner 时再检查有效 GID 与附加组；都未命中才使用 other。命中一类后不会把另外两类相加，也不会因为该类权限不足而回退到更宽松的一类。

概念

普通文件的 `rwX` 分别控制读取内容、修改现有内容和直接执行。文件自身的 `w` 不决定能否删除文件名，因为名称属于父目录；文件有 `r` 也不代表路径一定可达。

概念

目录的 `rwX` 分别控制枚举名称、修改目录项和 search/穿越。访问深层文件时，路径中的每一级目录都需要目标主体所选权限类的 `X`；创建、删除和重命名通常要求父目录的 `w+X`。

概念

`umask` 是进程持有的创建屏蔽状态，不是已有对象的默认 mode，也不是普通减法。最终传统权限等于程序请求模式按位清除 `mask` 中的位；它只能删除请求中的权限，不能凭空增加权限。

概念

`setuid`、`setgid` 与 `sticky` 是位于普通 `rwX` 之外的特殊位。`setuid`/`setgid` 可执行文件影响执行后进程的有效身份；`setgid` 目录影响新对象的 `group` 继承；`sticky` 目录限制公共可写目录中的删除与重命名。必须结合对象类型和相应执行位一起解释。

操作语义

以下入口分别观察对象元数据、修改 `mode`、修改 `owner/group`、控制新对象权限、展开路径和切换验证主体。先理解命令作用对象，再记关键形式。

ls / stat

SYNOPSIS

```
ls [OPTION]... [FILE]...
stat [OPTION]... FILE...
```

快速读取对象类型、符号权限、八进制 `mode`、`owner` 与 `group`；它们证明静态元数据，不证明目标用户一定能完成原始动作。

重要参数 / 形式

```
ls -ld DIR
```

查看目录本身，而不是列出目录内容。

```
ls -la DIR
```

包含隐藏名称，适合盘点目录项。

```
stat -c FORMAT PATH
```

固定输出 ``%A %a %U:%G %u:%g %n``，便于变更前后比较。

chmod

SYNOPSIS

```
chmod [OPTION]... MODE[,MODE]... FILE...
```

修改 `mode bits`。符号模式适合在现状上做最小增量，八进制适合写出精确终态。

重要参数 / 形式

```
u/g/o/a + - = rwxXst
```

显式选择权限类、操作符与权限位。

```
0640 / 0750
```

普通文件与目录的完整八进制终态。

```
2770 / 1777
```

首位表达 setgid 或 sticky 等特殊位。

-R

递归修改，执行前必须盘点对象类型、符号链接和文件系统边界。

X

只对目录或原本已有执行位的对象添加执行/search 位。

chown / chgrp

SYNOPSIS

```
chown [OPTION]... [OWNER][:[GROUP]] FILE...
chgrp [OPTION]... GROUP FILE...
```

修改 inode 的 owner UID 或 group GID；它们不自动修改普通 **rwX**，也可能触发 setuid/setgid 被清除。

重要参数 / 形式

chown OWNER:GROUP PATH

同时设置 owner 与 group。

chown :GROUP PATH

只修改 group。

chgrp GROUP PATH

用更直接的命令表达“只改 group”。

-R

递归改变所有权；先确认绝对路径、挂载点、链接和对象集合。

umask

SYNOPSIS

```
umask [-p] [-S] [MODE]
```

查看或改变当前 Shell 进程及其后代的创建屏蔽状态。它不修改已有对象，持久性必须通过新会话验证。

重要参数 / 形式

umask

显示当前数值 mask。

umask -S

以符号形式显示允许保留的权限。

umask 0007

当前 Shell 及其后代屏蔽 other 的全部普通权限。

requested & ~umask

正确的按位清除模型。

namei

SYNOPSIS

```
namei [OPTIONS] PATH...
```

把完整路径拆成每一级分量，适合寻找第一处缺少目录 search 权限的阻断点。

重要参数 / 形式

```
namei -l PATH
```

逐级显示对象类型、符号权限、owner 与 group。

从上到下判断

结合目标用户的 `id`，确认每一级选中 owner/group/other 哪一类。

第一阻断点

优先修复第一处有区分度的问题，不一次修改多级目录。

```
sudo -u / id
```

SYNOPSIS

```
sudo -u USER [--] COMMAND [ARG]...  
id [USER]
```

在本章只用于建立目标普通用户的测试进程并观察其凭据；sudoers 规则本身属于第 10 章。

重要参数 / 形式

```
id USER
```

查看账号数据库解析出的 UID、GID 与组。

```
sudo -u USER id
```

观察新建测试进程实际携带的身份。

```
sudo -iu USER bash -lc 'COMMAND'
```

需要登录式 Bash 环境时，通过目标用户的登录 Shell 执行命令；最终仍应按题目提供的真实登录方式复核。

正向 + 负向

目标用户完成原始动作，无关用户保持失败。

知识专题 从主体到 inode：传统 DAC 如何选择权限类

权限题的第一步不是换算八进制，而是确定访问主体和最终会被选中的权限类。只有把“进程身份”“对象所有权”和“操作类型”分开，才不会出现 owner 位不足时错误地继续借用 group 或 other 权限。

① 知识点 访问主体是进程凭据，不是屏幕上的用户名

每个进程都携带有效用户 ID、有效组 ID 和附加组集合。文件访问通常依据这些有效身份判断。`id USER` 查询账号数据库中该用户应具有的身份；在目标用户会话中执行 `id`，才能观察该进程当前实际携带的身份。管理员刚把用户加入新组时，旧登录会话可能仍保留旧的附加组集合，因此“数据库中已经是组成员”和“当前进程已经获得组身份”必须分开验证。

```
id alice          # 查询 alice 应有的 UID/GID 与组
sudo -u alice id  # 以目标身份建立测试进程并查看身份
```

本章假定用户和组已经由第 07 章《用户、组与账号生命周期》建立，不重复展开账号创建和组成员维护。

② 知识点 owner、group、other 三类只选择一类

对一个对象进行传统权限检查时，可以使用以下判断顺序：

```
访问进程的有效 UID 等于对象 owner UID?
├ 是：只使用 owner 位
└ 否：进程的有效 GID 或附加组命中对象 group GID?
    ├── 是：只使用 group 位
    └ 否：使用 other 位
```

“最具体的匹配优先”不等于“权限不足时继续回退”。例如某文件为：

```
-r--rw----  alice project report.txt
```

`alice` 同时属于 `project` 组时，仍只使用 owner 的 `r--`，不能把 group 的 `rw-` 加到 owner 上，也不会在 owner 缺少写权限后继续尝试 group。

③ 知识点 权限判断必须先明确操作

“能否访问文件”不是一个可判定问题。需要先改写成具体动作：

- 读取文件内容；
- 修改现有内容；
- 直接执行文件；
- 列出目录中的名称；
- 穿越目录并访问已知名称；
- 在目录中创建新条目；
- 删除或重命名目录项。

相同 mode 对不同动作可能给出不同结果。排错时应复现原始动作，而不是只执行 `ls` 或只执行 `test -r`。

④ 知识点 root 与 capabilities 是普通 DAC 模型的边界

特权进程可以绕过许多普通 DAC 检查，但这不代表 mode 无意义，也不能用 root 测试代替普通用户验收。对普通可执行文件，即使是特权进程，在没有任何执行位时也不能简单把它当作普通直接执行目标；挂载选项、解释器、SELinux 等也可能继续限制执行。

考试和日常变更中，默认应以题目指定的普通用户验证，不把“root 成功”扩大为业务终态正确。

[Cheatsheet] `id USER` 看账号应有身份；目标会话中的 `id` 看当前进程身份；owner/group/other 只选一类，不相加、不回退；先把“访问”改写成读取、写入、穿越、创建、删除或执行。

知识专题 文件与目录的 `rwX`：同一字母控制不同能力

普通文件保存内容，目录保存“名称到 inode”的映射，因此相同的 `rwX` 在两类对象上的含义不同。权限诊断必须同时检查最终对象和父目录；很多“文件权限正确但仍失败”的问题，实际阻断点在路径中间的目录。

① 知识点 普通文件的 `r`、`w`、`x`

`r` 读取内容

允许读取普通文件的数据；它不证明父路径可穿越。

`w` 修改内容

允许修改或截断现有内容；删除文件名主要看父目录。

`x` 直接执行

允许把文件作为程序启动；格式、解释器、挂载和其他安全层仍需成立。

脚本即使可以被解释器显式读取，例如 `bash script.sh`，也不等同于脚本本身具有直接执行权限。题目要求“可执行”时，应按题意验证直接执行路径，而不是用解释器绕过评分对象。

② 知识点 目录的 `r`、`w`、`x`

`r` 枚举名称

允许读取目录项名称；没有 `x` 时，很多条目元数据仍无法取得。

`w` 修改目录项

允许创建、删除和重命名名称；通常必须与 `x` 配合。

`x` search / 穿越

允许访问已知名称并继续路径解析；即使没有 `r`，知道准确名称时仍可能访问。

常见组合：

- `r-x`：可列出并访问，但不能创建或删除；
- `--x`：不能枚举名称，知道准确名称时可继续访问；
- `r--`：可看到名称，但不能可靠取得条目元数据或进入子路径；
- `-wx`：可以在已知目录中创建、删除和改名，但不能正常列出全部名称；这是高风险而少见的组合。

③ 知识点 路径中的每一级目录都需要 x

访问 `/srv/team/reports/q1.txt` 不只检查 `q1.txt` :

```
/                需要 search
/srv             需要 search
/srv/team        需要 search
/srv/team/reports 需要 search
q1.txt           根据动作需要 r、w 或 x
```

任一父目录缺少目标主体所选权限类的 `x`，路径解析都会在该处停止。最终文件即使是 `0644`，也可能无法读取。

④ 知识点 创建、删除和重命名首先由父目录控制

文件名属于父目录的目录项。创建新文件需要父目录的 `w+x`；删除或重命名现有文件也主要需要父目录的 `w+x`，而不是文件自身的 `w`。因此：

- 只读文件可能被拥有父目录 `w+x` 的用户删除；
- 可写文件若父目录不可写，用户可以改内容但不能删除名称；
- sticky 位可进一步限制公共可写目录中的删除和重命名。

删除目录还必须满足对象类型和空目录等额外条件；本章关注权限控制点，不把命令自身的结构条件混同为权限。

⑤ 知识点 目录权限可以屏蔽更深层对象

传统权限通常不自动从父目录复制到已有子对象，但父目录可以通过路径穿越能力屏蔽其内部所有内容。不能因为子文件是 `0644` 就推断所有用户可读，也不能因为目录是 `0755` 就推断里面所有文件可读。

[Cheatsheet] 文件：`r` 读内容、`w` 改内容、`x` 直接执行；目录：`r` 枚举名称、`w` 改目录项、`x` 穿越；创建/删除/改名看父目录 `w+x`；路径每一级都要 `x`。

操作专题 使用 `ls`、`stat`、`id` 与 `namei` 建立权限证据

查询工具的职责不同。`ls -l` 适合快速阅读，`stat` 适合精确取值，`id` 确认主体，`namei -l` 则把路径逐级展开。权限排错应组合使用，而不是把任何一条命令当作完整证明。

① 操作 用 `ls -l` 和 `ls -ld` 区分对象与目录内容

作用对象：指定路径或目录内容。基本形式：

```
ls -l FILE
ls -ld DIR
ls -la DIR
```

`ls -l DIR` 默认列出目录内容；要看目录本身，应使用 `-d`。长格式的首字符表示类型，之后九位依次为 owner、group、other 的 `rwX`。例如：

```
drwxrws---
| | | |
| | | |  other
| | | |  group; 执行位位置为 s, 表示 setgid 与 x 同时存在
| | | |  owner
| | | |  目录
```

权限串末尾可能出现 `+` 或 `.`。它们提示还存在 ACL 或安全上下文等扩展信息，但不在本章展开；出现这类标记时，应把它作为转入第 09 章 ACL 或第 28/29 章 SELinux 的信号，而不是忽略。

② 操作 用 `stat` 同时读取符号与八进制状态

作用对象：单个文件系统对象。典型形式：

```
stat PATH
stat -c '%F | %A | %a | %U:%G | %u:%g | %n' PATH
```

关键字段：

- `%F`：对象类型；
- `%A`：符号权限；
- `%a`：八进制权限；
- `%U:%G`：名称形式 owner/group；
- `%u:%g`：数字 UID/GID；
- `%n`：路径名。

名称解析异常时，数字 UID/GID 比显示名称更接近 inode 真相。变更前后使用相同格式，便于形成可比较基线。

③ 操作 用 `id` 区分账号记录与当前进程身份

```
id alice
sudo -u alice id
```

第一条查询系统为 `alice` 解析出的身份；第二条建立实际测试进程。若题目刚修改过组成员，真实登录会话可能还需要重新建立，不能仅依据 `id alice` 推断旧会话已刷新。

④ 操作 用 `namei -l` 逐级展开路径

作用对象：完整路径中的每个分量。典型形式：

```
namei -l /srv/team/reports/q1.txt
```

调查时从上到下寻找第一处目标主体缺少 `x` 的目录。`namei` 显示的是静态 mode 与 owner/group；仍需结合 `id` 判断该用户在每一级将选中 owner、group 还是 other。

⑤ 输出判断 `s/S/t/T` 同时编码特殊位和执行位

`-rwsr-xr-x` owner 的执行位位置：setuid 与 owner `x` 同时存在。

`-rwSr-xr-x` owner 的执行位位置：setuid 存在，但 owner `x` 缺失。

`-rwxr-sr-x` group 的执行位位置：setgid 与 group `x` 同时存在。

`-rwxr-Sr-x` group 的执行位位置：setgid 存在，但 group `x` 缺失。

`drwxrwxr-t` other 的执行位位置：sticky 与 other `x` 同时存在。

`drwxrwxr-T` other 的执行位位置：sticky 存在，但 other `x` 缺失。

大写字符不是“更强”，反而通常提示特殊位存在但相应执行位缺失，需要核对是否符合目标。

⑥ 验证 查询只能证明元数据，功能必须实际测试

`stat` 显示 `0660` 不能证明 bob 能写：bob 可能没有当前组身份，路径中间可能缺少 `x`，也可能还有 ACL 或 SELinux 限制。标准验证顺序为：

```
stat/namei/id 建立证据
→ 以目标用户执行原始动作
→ 以非授权用户执行负向动作
```

[Cheatsheet] 看目录本身用 `ls -ld`；精确 mode/UID/GID 用 `stat -c`；主体用 `id`；完整路径用 `namei -l`；`s/S/t/T` 要同时看特殊位和执行位；元数据正确后仍要实际用户测试。

操作专题 使用 `chmod` 表达增量修改与精确终态

`chmod` 改变的是 mode bits。符号模式适合最小增量修改，八进制适合把对象设置为明确终态。选择哪一种，应由题目是“在现状上添加/删除”还是“最终必须等于某模式”决定。

① 操作 符号模式的结构

基本形式：

```
chmod [ugoa][+=[-]] [rwxXst] PATH
```

- `u`：owner；`g`：group；`o`：other；`a`：全部；
- `+`：添加；`-`：删除；`=`：用给定集合替换该类权限；
- `rwx`：普通权限；`X`：条件执行/search；`s`：setuid/setgid；`t`：sticky。

典型操作：

```
chmod g+w report.txt      # 只添加 group write
chmod o-rwx secret.txt    # 删除 other 的全部普通权限
chmod u=rw,g=r,o= file.txt # 精确替换三类普通权限
chmod g+s /srv/project    # 给目录添加 setgid
chmod +t /srv/dropbox     # 给目录添加 sticky
```

在脚本和考试答案中建议显式写出 `u/g/o/a`。省略 `who` 时，当前 `umask` 可能影响哪些类别被修改，容易让相同命令在不同会话产生不同终态。

② 操作 八进制模式写出精确权限

每一类权限按 `r=4`、`w=2`、`x=1` 求和：

```
7 = rwx
6 = rw-
5 = r-x
4 = r--
0 = ---
```

```
chmod 0640 report.txt
chmod 0750 scripts
chmod 2770 /srv/project
chmod 1777 /srv/dropbox
```

当特殊位属于评分终态时，使用四位形式更清楚：首位 `4=setuid`、`2=setgid`、`1=sticky`。八进制命令表达“整个模式的目标值”，执行前要确认不会意外删除本应保留的权限。

③ 参数 大写 `x` 适合目录树，不等于无条件 `x`

`X` 仅在对象是目录，或者对象原本任一类别已有执行位时添加执行/`search` 权限。典型用途：

```
chmod -R g+rwX /srv/project
```

它会给目录添加 `group search`，并为目录树中的对象添加 `group read/write`；普通、原本完全不可执行的文件不会仅因递归而被变成可执行文件。但若某文件原本任何类别已有 `x`，`X` 可能继续给指定类别添加执行位，因此仍要先盘点对象。

④ 操作 复制权限类时使用 `u/g/o` 作为权限来源

符号模式的权限部分也可以引用另一类：

```
chmod g=u file      # 让 group 普通权限等于 owner
chmod o=g file      # 让 other 普通权限等于 group
```

这适合“让某类与另一类一致”的任务，但不自动复制 owner/group 身份，也不等同于 ACL。

⑤ 验证 修改后同时检查符号、八进制与功能

```
chmod 2770 /srv/project
stat -c '%A %a %U:%G %n' /srv/project
```

随后必须以目标身份创建或访问对象。对目录而言，仅验证 2770 只能证明目录 mode，不能证明新文件具有目标组和组写权限。

⑥ 安全边界 不把 chmod 777 当作诊断方法

777 同时向所有本地主体开放读、写、执行或目录修改能力，会掩盖 owner/group、umask、路径、ACL、SELinux 等真正问题。推荐链路：

```
先定位哪一级、哪一类、缺哪一位
→ 做最小修改
→ 重新执行原始操作
```

[Cheatsheet] 增量用符号模式，精确终态用八进制；特殊位写四位；递归目录树优先考虑 X 而非无条件 x；显式写 who；变更后 stat + 实际用户测试；不使用 777 碰运气。

操作专题 使用 chown 与 chgrp 修改 owner 和 group

传统 mode 只有在正确的 owner/group 关系下才会选中预期权限类。修改所有权和修改权限是两种不同操作：chmod 不改变 UID/GID，chown / chgrp 也不自动补齐 rwx。

① 操作 chown 的常用形式

```
chown alice FILE          # 只改 owner
chown :project FILE       # 只改 group
chown alice:project FILE  # 同时改 owner 与 group
```

推荐使用冒号分隔 owner 与 group。点号可能是合法用户名的一部分，使用旧式 owner.group 容易产生歧义。

② 操作 chgrp 只改变 group

```
chgrp project FILE
chgrp -R project DIR
```

chgrp project FILE 与 chown :project FILE 的目标相同。选用哪条命令可依据可读性和任务上下文，但操作后都应使用 stat 核对 GID。

③ 权限边界 普通用户不能任意转让 owner

通常只有特权用户可以把文件 owner 改成其他用户。文件 owner 可以把 group 改成自己所属的某个组；特权用户可以设置为任意有效组。考试任务若要求确定 owner/group，通常以管理员身份执行，再以普通用户验证。

④ 边界 所有权变化可能清除 setuid/setgid

为防止权限提升，内核或工具在改变 owner/group、写入可执行文件等操作后可能清除 setuid/setgid 位。任何涉及特殊位的对象在 `chown`、`chgrp` 或内容变更后，都应重新执行：

```
stat -c '%A %a %U:%G %n' PATH
```

不要假定之前设置的 `4755`、`2755` 必然保留。

⑤ 安全 递归所有权变更要明确链接与边界

`chown -R` 能在很短时间内改变整个目录树。执行前至少确认：

- 展开的绝对路径是否正确；
- 目录是否为挂载点或包含其他文件系统；
- 树中是否存在符号链接；
- 是否真的所有子对象都应改变；
- 变更后哪些服务或用户依赖旧 owner/group。

可先使用 `find` 或 `stat` 建立清单，再做限定范围的修改。不要对不确定路径执行宽泛递归。

[Cheatsheet] `chown USER` 改 owner；`chown :GROUP` 或 `chgrp GROUP` 改 group；`chown USER:GROUP` 同时改；普通用户不能任意转让 owner；所有权或内容变化后重查特殊位；递归前先确认路径、挂载和符号链接。

知识专题 `umask`：新对象权限的按位清除模型

`umask` 不是“默认权限值”，也不是对已有对象执行的 `chmod`。它是进程状态，用于清除创建请求中的权限位。掌握按位模型，可以避免把八进制当普通十进制做减法，也能解释为什么同一个 `umask` 下不同程序仍可能创建出不同权限。

① 知识点 正确公式是请求模式按位清除 mask

```
最终普通权限 = 程序请求模式 AND (NOT umask)
```

常见命令行工具通常请求：

- 普通文件：`0666`，默认不请求执行位；

- 目录：0777。

例如 `umask 0027`：

```
文件：0666 & ~0027 = 0640
目录：0777 & ~0027 = 0750
```

② 知识点 umask 只能删除权限，不能增加权限

如果程序主动请求 0600，即使 umask 为 0000，结果也不会变成 0666。同理，普通文件的常见请求不含执行位，因此 `umask 0000` 通常也不会让新普通文件自动可执行。

③ 知识点 不能把 umask 当普通减法

`0666 - 0027` 在某些例子上看似得到正确结果，但它不是权限算法，遇到重叠位时会误导。例如请求模式本来没有某位时，mask 不能从别的位置“借位”。学习和排错应逐位清除：

```
请求  rw-rw-rw-
mask  ----w-rwx
结果  rw-r-----
```

④ 知识点 umask 属于进程并由子进程继承

在当前 shell 执行 `umask 0007` 后，由该 shell 启动的命令继承相同 mask，除非程序主动修改。退出该 shell 后，父进程或新登录流程可能提供另一值。因此：

- 当前 shell 中看到的值是当前状态；
- 配置文件中写入的是策略来源；
- 新登录会话中看到的值才是持久性证据；
- 新建样本对象的 mode 才是功能证据。

⑤ 边界 default ACL 会改变创建权限路径

如果父目录存在 default ACL，新对象会先从 default ACL 继承访问 ACL，再受创建请求中的权限限制；此时不能只按简单 umask 表推断最终有效权限。发现 `ls -ld` 末尾有 + 或 `getfacl` 显示 default 条目时，应转入第 09 章《ACL 与协作目录》。本章只建立接口，不展开 ACL mask 计算。

⑥ 计算表 常见 umask 的典型结果

0022 文件 0644 · 目录 0755

owner 可写，其他只读或穿越。

0027 文件 0640 · 目录 0750

组可读/穿越，other 隔离。

0007 文件 0660 · 目录 0770

owner 与 group 协作，other 隔离。

仅 owner。

表格是假定程序请求 0666/0777 的典型结果，不替代实际创建与 stat。

[Cheatsheet] umask 是进程状态；公式是 `requested & ~mask`；只能清除不能授予；文件常见起点 0666、目录 0777；当前值、新会话值和新对象结果分层验证；default ACL 存在时转下一章。

操作专题 设置并验证当前与持久 umask

持久 umask 不是“在任意配置文件末尾写一行”这么简单。必须先明确目标主体、Shell 类型和会话入口，再修改对应来源，并用新会话验证。考试题若只要求当前任务进程的创建结果，显式在同一命令环境中设置反而更可控。

① 操作 查看与临时设置

```
umask          # 数值形式，常见输出如 0022
umask -S       # 符号形式，显示允许的权限
umask 0007     # 只改变当前 shell 及其后代
```

注意：umask -S 表示最终允许保留的权限类，而不是直接打印 mask 数字，阅读时不要把两种输出混为一谈。

② 验证 创建文件和目录样本

```
umask 0007
rm -f /tmp/umask-file
rm -rf /tmp/umask-dir
touch /tmp/umask-file
mkdir /tmp/umask-dir
stat -c '%A %a %U:%G %n' /tmp/umask-file /tmp/umask-dir
```

在没有 default ACL 且工具使用常见请求模式时，期望文件为 0660、目录为 0770。验证后清理测试对象，避免旧样本干扰下一次测试。

③ 配置 选择与目标会话匹配的启动文件

以 Bash 为例，登录 shell 与非登录交互 shell 的读取路径不同。可按题目范围选择：

- 用户登录 shell 策略：在用户登录启动文件中设置；
- 用户交互 Bash 策略：在用户的 Bash 初始化文件中设置；
- 系统范围：使用明确的系统 profile/bashrc 或 PAM 策略，并评估对现有账号影响。

不要宣称 /etc/login.defs、/etc/profile 或 ~/.bashrc 中任一处必然覆盖所有服务、计划任务和非交互程序。systemd service、容器和应用自身也可能显式设置创建模式。

④ 验证 新会话与创建结果缺一不可

建议验收矩阵：

```
配置文件存在且语法正确
→ 新建目标登录会话
→ 新会话执行 umask
→ 在该会话创建文件和目录
→ stat 检查最终 mode
```

只查看配置文件不能证明它被读取；只查看 `umask` 不能证明程序请求模式；只查看样本不能说明持久来源正确。

⑤ 边界 协作目录不应只依赖每个人手工执行 umask

多人协作若要求稳定组写权限，`setgid` 负责组继承，合适 `umask` 负责普通创建上限；但不同入口可能具有不同 `mask`。若业务必须对多种程序和入口统一继承权限，应在第 09 章评估 default ACL，而不是假定所有用户都会保持同一 shell `umask`。

[Cheatsheet] 临时：`umask NNNN`；查看：`umask / umask -S`；验证：新文件 + 新目录；持久性：必须新建会话；先明确登录/交互/服务入口，再选配置来源；协作要求跨入口稳定时转 ACL。

知识专题 setuid、setgid 与 sticky：特殊位改变哪一层语义

特殊位不等于“额外的 `rwX`”。`setuid` 和 `setgid` 主要影响执行后进程的有效身份；`setgid` 还对目录提供组继承；`sticky` 则改变公共可写目录中的删除和重命名规则。判断时必须结合对象类型。

① 知识点 八进制首位 4/2/1

```
4 = setuid
2 = setgid
1 = sticky
```

可以组合，例如首位 `6` 同时包含 `setuid` 和 `setgid`。常见完整模式：

```
4755  setuid 可执行文件
2755  setgid 可执行文件
2770  setgid 组协作目录
1777  sticky 公共可写目录
```

② 知识点 setuid 作用于可执行文件的有效 UID

具有 `setuid` 的可执行文件被允许执行时，进程的有效 UID 通常取文件 `owner`，而真实 UID 仍表示发起用户。它用于让受控程序完成普通用户本来无权直接完成的特定操作。

安全边界：

- `setuid` 不是给普通脚本随意提权的方案；Linux 通常忽略脚本的 `setuid` 语义；
- 可执行文件必须由可信主体控制，不能让非特权用户修改；
- 内容或所有权变化后应重新检查特殊位；
- 不在本章展开 `capabilities` 和应用安全审计。

③ 知识点 `setgid` 对可执行文件改变有效 GID

`setgid` 可执行文件运行后，进程的有效 GID 通常取文件 `group`。它与 `setuid` 类似，但改变的是组身份。该机制不是 `setgid` 目录组继承的同义词，必须根据对象是“可执行文件”还是“目录”分别解释。

④ 知识点 `setgid` 目录让新对象继承目录 `group`

在 Linux 上，`setgid` 目录中的新文件通常继承目录 `group`，而不是创建进程的主组；新建子目录通常还会继承 `setgid` 位，从而继续保持组归属链。

它只解决“新对象属于哪个组”，不自动保证：

- `group` 拥有写权限；
- 已有对象改成目标组；
- 用户当前已经获得该组身份；
- ACL 或 SELinux 允许操作。

因此 `chmod 2770 DIR` 只是协作目录的一层。

⑤ 知识点 `sticky` 限制公共可写目录中的删除和重命名

在可写目录上设置 `sticky` 后，即使用户有目录 `w+x`，通常也只能删除或重命名：

- 自己拥有的目录项；
- 目录 `owner` 拥有的管理范围；
- 自己是文件 `owner` 的对象；
- 特权主体允许的对象。

典型公共临时目录为 `1777`：所有用户可以创建，但不能随意删除其他用户的文件。`sticky` 不阻止读取文件内容；文件本身的 `rwX` 仍单独判断。

⑥ 输出判断 大写 `S/T` 是警报，不是增强

```
-rwSr-xr-x   setuid 已设置，但 owner 没有 x
-rwxr-Sr-x   setgid 已设置，但 group 没有 x
-drwxrwxr-T  sticky 已设置，但 other 没有 x
```

对可执行文件，缺少对应执行位往往意味着特殊执行身份无法按预期生效；对目录，`T` 还提示 `other` 不能穿越。看到大写字符应回到题目终态，不要机械认为“特殊位已经有了就正确”。

⑦ 边界 特殊位的目录/文件语义并不对称

- `setuid` 在 Linux 目录上通常没有通用的 owner 继承意义；
- `setgid` 在目录上具有重要组继承语义；
- `sticky` 在目录上用于删除/重命名限制，普通文件上的历史语义不属于 RHCSA 日常操作。

[Cheatsheet] 首位 4/2/1 = `setuid/setgid/sticky`；`setuid/setgid` 可执行文件改有效 UID/GID；`setgid` 目录继承 group；`sticky` 目录限制互删；`s/t` 含执行位，`S/T` 缺执行位；特殊位不替代普通 `rwX`。

操作专题 构造不使用 ACL 的 `setgid` 组协作目录

传统权限可以实现“一个固定组共同工作、other 完全隔离”的基本协作目录。完整方案至少包含目录 group、`setgid`、组 `rwX`、创建进程的 `umask`，以及两个组成员之间的交叉验证。

① 调查 先确认目录、组和测试主体

假定组和用户已存在：

```
getent group techdocs
id alice
id bob
id carol
stat -c '%F %A %a %U:%G %n' /srv/techdocs 2>/dev/null || true
namei -l /srv/techdocs
```

目标：alice、bob 是 `techdocs` 成员，carol 不是；目录 owner 为 root，group 为 `techdocs`，other 无权限。

② 操作 设置目录归属和 mode

```
install -d -o root -g techdocs -m 2770 /srv/techdocs
```

已有目录应先调查再分别使用 `chown root:techdocs` 与 `chmod 2770`，避免 `install -d` 的简洁形式掩盖已有内容状态。操作后：

```
stat -c '%A %a %U:%G %n' /srv/techdocs
```

期望静态终态为 `drwxrws--- 2770 root:techdocs`。

③ 配置 为创建进程提供组可写的 `umask`

在不使用 default ACL 的前提下，创建者需要使用允许 group write、屏蔽 other 的 mask，例如 `0007`：

```
sudo -u alice sh -c 'umask 0007; touch /srv/techdocs/alice.txt; mkdir
/srv/techdocs/alice.d'
```

若题目要求持久策略，应在明确的登录环境中配置，再用新会话验证；不能只在这一条测试命令中设置。

④ 验证 检查组继承和最终模式

```
stat -c '%A %a %U:%G %n' \  
/srv/techdocs/alice.txt /srv/techdocs/alice.d
```

在典型创建请求下，期望：

```
alice.txt 0660 alice:techdocs  
alice.d   2770 alice:techdocs
```

子目录保留 setgid，才能让更多新对象继续继承组。

⑤ 验证 第二名组员实际修改

```
sudo -u bob sh -c 'printf "%s\n" reviewed >> /srv/techdocs/alice.txt'  
sudo -u bob test -w /srv/techdocs/alice.txt
```

`test -w` 是快速检查，实际追加才是更强的功能证据。失败时依次检查 bob 当前组、父路径 `x`、文件 group、文件 group write、ACL/SELinux。

⑥ 负向验证 非成员不能穿越或创建

```
sudo -u carol test -x /srv/techdocs  
sudo -u carol touch /srv/techdocs/should-fail
```

负向命令预期失败。测试文件名必须是专用样本，避免破坏真实数据。

⑦ 边界 setgid 不回溯修复已有对象

目录加 setgid 后，已有文件不会自动改组、补写权限或获得 setgid。已有树若也必须迁移，应先盘点 owner/group/mode，再使用受控的 `chgrp`、`chmod` 或限定 `find` 操作；这属于一次独立变更，不能被“新对象继承”替代。

[Cheatsheet] `root:GROUP + 2770` 建立目录层；setgid 管新对象 group；`umask 0007` 让典型新文件/目录组可写且隔离 other；验证用 A 创建、B 修改、非成员失败；已有对象单独迁移。

诊断专题 文件显示可读却仍 `Permission denied`

这类故障最适合训练证据推进。最终文件 mode 只是链路末端的一项；正确方法是从真实主体开始，逐级找到第一处具有区分度的阻断点，再做最小修复。

① 症状 固定原始动作和目标主体

例如：

```
sudo -u dana cat /srv/reports/q1/report.txt
```

不要先改权限。记录：谁执行、完整路径、动作是读内容而不是列目录、错误发生在当前系统还是远程程序中。

② 当前证据 确认主体身份

```
id dana
sudo -u dana id
```

若组成员关系刚改变，应建立新会话。不要用管理员自己的 `id` 代表 `dana`。

③ 下一条最有区分度的证据 展开路径

```
namei -l /srv/reports/q1/report.txt
```

逐行判断 `dana` 在每个目录上选中 `owner/group/other` 哪一类，找到第一处缺少 `x` 的分量。若路径均可穿越，再检查最终文件 `r`。

④ 假设 区分父目录、最终文件与删除语义

- `cat` 失败：父路径 `x` 或文件 `r`；
- `echo >> file` 失败：父路径 `x` 或文件 `w`；
- `touch NEW` 失败：父目录 `w+x`；
- `rm file` 失败：父目录 `w+x`，以及 `sticky` 等额外限制；
- `ls DIR` 失败：目录 `r/x` 组合。

选择与原始症状最相关的假设，不要一次改多层。

⑤ 最小修复 只修改阻断对象和目标权限类

如果 `/srv/reports` 的 `group` 为 `analysts`，`dana` 属于该组，但目录 `mode` 为 `0740`，缺的是 `group search`。可在确认业务终态后执行：

```
chmod g+x /srv/reports
```

而不是把最终文件改成 `777`。修改后立即用 `stat` 和 `namei` 重建证据。

⑥ 再验证 用相同主体重做原始动作

```
sudo -u dana cat /srv/reports/q1/report.txt
```

再使用一个无关用户做负向测试，确保最小修复没有扩大给 other。

⑦ 边界 DAC 链路无异常后再进入下一安全层

若 `id`、`namei`、`stat` 均支持传统权限允许，但操作仍被拒绝，下一步依次检查：

- 第 09 章：访问 ACL、default ACL 和 mask；
- 第 28/29 章：SELinux 上下文、类型规则和 AVC 证据；
- 应用自身的访问控制或只读挂载等其他层。

[Cheatsheet] 固定主体和原始动作 → `id` → `namei -l` → `stat` → 判断选中的 u/g/o → 最小 `chmod/chown` → 相同用户复验 → 无关用户负向测试 → 再转 ACL/SELinux。

诊断专题 递归权限修改：先控制爆炸半径

递归命令会把一个小判断错误扩散到整棵树。`chmod -R 777`、`chown -R` 或对错误变量展开执行，可能破坏可执行文件、安全边界和服务数据。递归不是禁用功能，而是必须先建立对象集合、链接策略和可验证终态。

① 调查 先盘点对象类型和现状

```
find /srv/project -xdev -printf '%y %m %u:%g %p\n' | less
find /srv/project -xdev -type l -print
```

`-xdev` 可在适用时避免跨入其他文件系统，但是否使用取决于任务边界。确认树中是否有脚本、二进制、套接字、命名管道、挂载点和符号链接。

② 假设 文件和目录通常需要不同权限

目录需要 `x` 才能穿越，普通数据文件通常不应自动获得执行位。若目标是“组成员可读写并穿越目录”，可考虑：

```
chmod -R g+rwX /srv/project
```

仍需确认原本可执行文件是否应保持执行，以及 other 权限是否需要收紧。

③ 边界 明确符号链接跟随策略

递归工具对命令行参数中的符号链接、树内符号链接以及 `-H/-L/-P` 的处理不同。默认不要假定链接会或不会跟随；执行前查看对应 man page，并尽量对真实根目录路径操作。链接指向树外时，跟随可能把变更扩散

到完全不同的位置。

④ 最小修复 按类型和条件限定

精确迁移常使用分开的 `find`：

```
find /srv/project -xdev -type d -exec chmod 2770 {} +
find /srv/project -xdev -type f -exec chmod 0660 {} +
```

这只是任务示例：给每个子目录 `setgid` 是否符合业务、是否存在应执行脚本，必须在执行前确认。不能把示例当成任何目录树的通用答案。

⑤ 验证 全量静态检查与抽样功能测试结合

```
find /srv/project -xdev -printf '%y %m %u:%g %p\n'
```

随后选择不同层级、不同 `owner`、不同对象类型进行跨用户测试。只检查根目录或随机一个文件不能证明整棵树正确。

[Cheatsheet] 递归前先 `find` 盘点；文件和目录分开；需要目录 `search` 时优先评估 `X`；明确 `-H/-L/-P` 与跨文件系统边界；按类型限定；变更后全量静态核对 + 多身份功能抽样。

经典任务 建立传统权限组协作目录并完成跨用户验收

环境与当前状态

系统已经存在：

- 组 `techdocs`；
- 用户 `alice`、`bob`，均为 `techdocs` 成员；
- 用户 `carol`，不是该组成员；
- `/srv` 可由所有用户穿越；
- `/srv/techdocs` 当前不存在；
- `alice` 与 `bob` 使用 `Bash` 登录，二人的 `~/.bash_profile` 已存在并会加载各自的 `~/.bashrc`，当前没有显式 `umask` 行。

本任务不允许使用 `ACL`；`sudo` 仅作为测试身份切换工具，不要求修改 `sudoers`。

目标终态

1. 创建 `/srv/techdocs`，`owner` 为 `root`，`group` 为 `techdocs`；
2. 目录模式为 `2770`；
3. `alice` 和 `bob` 的新登录 `Bash` 会话应使用 `umask 0007`；

4. alice 新建的普通文件应为 `0660 alice:techdocs`；
5. alice 新建的子目录应为 `2770 alice:techdocs`；
6. bob 能向 alice 创建的文件追加内容；
7. carol 不能穿越目录，也不能创建文件；
8. 不修改系统中其他用户的 `umask`，不使用 `chmod 777`。

验收证据

- `stat` 显示目录及样本对象的 `mode` 和 `owner/group`；
- 新登录会话中的 `umask`；
- alice 创建、bob 修改的实际操作；
- carol 的负向测试；
- 必要时用 `namei -l` 证明父路径没有额外阻断。

参考解答 经典任务一

以下命令是静态推荐解答，必须在真实 RHEL 9 环境中再次验证用户启动文件、实际组身份和创建结果。

① 调查现有身份与路径

```
getent group techdocs
id alice
id bob
id carol
namei -l /srv
```

确认 alice、bob 命中 `techdocs`，carol 不命中；确认 `/srv` 的父路径允许目标用户穿越。

② 创建目录并设置精确终态

```
install -d -o root -g techdocs -m 2770 /srv/techdocs
stat -c '%A %a %U:%G %n' /srv/techdocs
```

若目录已经存在，应先备份基线并分别执行最小变更，而不是假定目录为空。

③ 配置目标用户的登录 `umask`

题目明确限定“新登录 Bash 会话”。应根据系统现有 Bash 启动结构，把 `umask 0007` 放入 alice、bob 的登录配置范围，同时避免重复和覆盖其他逻辑。示意：

```
for user in alice bob; do
    home=$(getent passwd "$user" | cut -d: -f6)
    profile="$home/.bash_profile"
    cp -a "$profile" "$profile.rhcsa08.bak"
    grep -qxF 'umask 0007' "$profile" ||
        printf '\numask 0007\n' >> "$profile"
done
```

本任务已明确这两个文件存在且属于目标用户，因此示例保留其所有权并先建立备份。真实系统仍应先查看启动链；若现有逻辑在其他文件统一设置，应修改实际生效来源，而不是机械追加多处。

④ 建立新登录会话验证当前状态

```
sudo -iu alice bash -lc 'umask'
sudo -iu bob bash -lc 'umask'
```

期望两者为 `0007`。`sudo -iu` 在此仅模拟登录式测试入口；最终以题目提供的真实登录方式复核更可靠。

⑤ alice 创建样本并检查继承

```
sudo -iu alice bash -lc 'touch /srv/techdocs/alice.txt; mkdir /srv/techdocs/alice.d'
stat -c '%A %a %U:%G %n' \
    /srv/techdocs/alice.txt /srv/techdocs/alice.d
```

期望：

```
alice.txt  0660 alice:techdocs
alice.d    2770 alice:techdocs
```

如果 group 正确但没有 group write，检查 alice 新会话的 umask；如果 group 不正确，检查父目录 setgid 和实际创建位置。

⑥ bob 进行交叉写入

```
sudo -iu bob bash -lc 'printf "%s\n" reviewed >> /srv/techdocs/alice.txt'
sudo -iu bob tail -n 1 /srv/techdocs/alice.txt
```

这比只看 `test -w` 更接近真实终态。

⑦ carol 做负向测试

```
sudo -u carol test -x /srv/techdocs
sudo -u carol touch /srv/techdocs/should-not-exist
```

两条命令预期失败。随后确认没有意外产生样本：

```
test ! -e /srv/techdocs/should-not-exist
```

⑧ 最终证据矩阵

维度	命令	证明
目录静态状态	<code>stat /srv/techdocs</code>	<code>2770 root:techdocs</code>
登录策略	<code>sudo -iu USER bash -lc 'umask'</code>	新登录会话使用 <code>0007</code>
组继承	<code>stat alice.txt</code>	新文件 group 为 <code>techdocs</code>
文件 mode	<code>stat alice.txt</code>	新文件为 <code>0660</code>
子目录延续	<code>stat alice.d</code>	子目录为 <code>2770</code> ，继续 <code>setgid</code>
组内功能	bob 追加	第二名成员能实际写入
隔离	carol 负向测试	other 没有穿越/创建能力

典型错误

- 只设置 `chmod 2770`，却不处理创建进程 `umask`；
- 只在当前管理员 shell 执行 `umask 0007`；
- 用 `root` 创建样本，误判普通用户终态；
- 只执行 `ls -ld`，没有跨用户创建和修改；
- 使用 `777` 或提前引入 `ACL`，偏离任务边界；
- 假定 `setgid` 会回溯修改已有文件。

经典任务 定位“文件可读但路径不可达”并做最小修复

环境与当前状态

系统存在用户 `dana`，她属于组 `analysts`。目标文件：

```
/srv/reports/q1/report.txt
```

管理员已确认最终文件静态状态为：

```
0640 root:analysts
```

但执行：

```
sudo -u dana cat /srv/reports/q1/report.txt
```

得到 `Permission denied`。不允许删除重建对象，不允许把文件或目录改成 `777`，不要求配置 ACL 或 SELinux。

目标终态

1. 找到路径中第一个传统权限阻断点；
2. 只增加 dana 读取该文件所需的最小权限；
3. dana 能读取 `report.txt`；
4. 不属于 `analysts` 的用户 `erin` 仍不能读取；
5. 提交调查、修改和再验证证据。

参考解答 经典任务二

① 固定主体和原始失败

```
id dana
sudo -u dana id
sudo -u dana cat /srv/reports/q1/report.txt
```

记录原始失败，不先改最终文件。

② 读取最终对象状态

```
stat -c '%F %A %a %U:%G %n' /srv/reports/q1/report.txt
```

`0640 root:analysts` 表明 dana 若当前组集合命中 `analysts`，最终文件的 group `r--` 足以读取内容；这仍不能证明路径可达。

③ 展开每一级路径

```
namei -l /srv/reports/q1/report.txt
```

对 `/`、`/srv`、`/srv/reports`、`/srv/reports/q1` 逐级判断 dana 选中的权限类是否包含 `x`。假定发现 `/srv/reports` 为：

```
drwxr----- root analysts /srv/reports
```

它的 group 有 `r` 但没有 `x`，因此 dana 可以命中 group，却无法穿越。

④ 做最小修改

```
chmod g+x /srv/reports
stat -c '%A %a %U:%G %n' /srv/reports
namei -l /srv/reports/q1/report.txt
```

修改只为 analysts 增加 search，不改变 other，不修改最终文件。

⑤ 以相同主体再验证

```
sudo -u dana cat /srv/reports/q1/report.txt
```

期望成功。

⑥ 非授权用户负向测试

```
id erin
sudo -u erin cat /srv/reports/q1/report.txt
```

期望失败。若 erin 仍可读取，应检查路径中 other 权限、她的组身份以及是否存在 ACL。

⑦ 典型错误

- 把 report.txt 从 0640 改成 0644，但父路径仍不可穿越；
- 把所有父目录加 o+x，扩大给无关用户；
- 看到目录有 r 就认为可穿越；
- 用 root 执行 cat 作为验收；
- 一次修改多个目录，无法判断真正阻断点；
- DAC 证据已经允许后仍继续乱改 mode，而没有转入 ACL/SELinux 证据链。

本章收束 用“主体—路径—权限类—动作—证据”解决权限题

传统权限的核心不是一组数字，而是一条求值链：

```
真实访问主体
→ 路径逐级 search
→ 每个对象选择 owner/group/other 唯一一类
→ 根据具体动作检查文件或目录 rwx
→ 应用 setuid/setgid/sticky 的对象类型语义
→ 对新对象应用 requested mode 与 umask
→ 实际用户正向与负向验证
```

最小证据矩阵

问题	首选证据
用户当前是谁、有哪些组	目标进程中的 <code>id</code>
对象当前 mode/owner/group	<code>stat -c</code>
路径哪一级不可达	<code>namei -l</code>
应增量还是精确设置	<code>chmod</code> 符号/八进制模式
owner/group 是否正确	<code>stat</code> + <code>chown/chgrp</code>
当前创建 mask	<code>umask</code> 、 <code>umask -S</code>
持久 mask 是否生效	新登录会话 + 创建样本
setgid 协作是否成功	A 创建、B 修改、非成员失败
<code>Permission denied</code> 下一层	ACL，再到 SELinux

工作迁移

在真实系统中，权限修改应视为变更而不是命令练习：先记录基线，明确业务主体和动作，控制递归范围，避免链接越界，保留回滚所需的 owner/group/mode 清单，并在操作后用真实身份验证。后续使用 Ansible 自动化时，也应把目标写成可比较的 mode、owner、group 和创建策略，而不是把一串无条件 shell 命令当作幂等状态。

本章完成了传统 DAC 层。额外用户/组授权、default ACL 与 mask 进入第 09 章《ACL 与协作目录》；sudo 授权进入第 10 章《sudo 与最小特权授权》；SELinux 进入第 28、29 章。